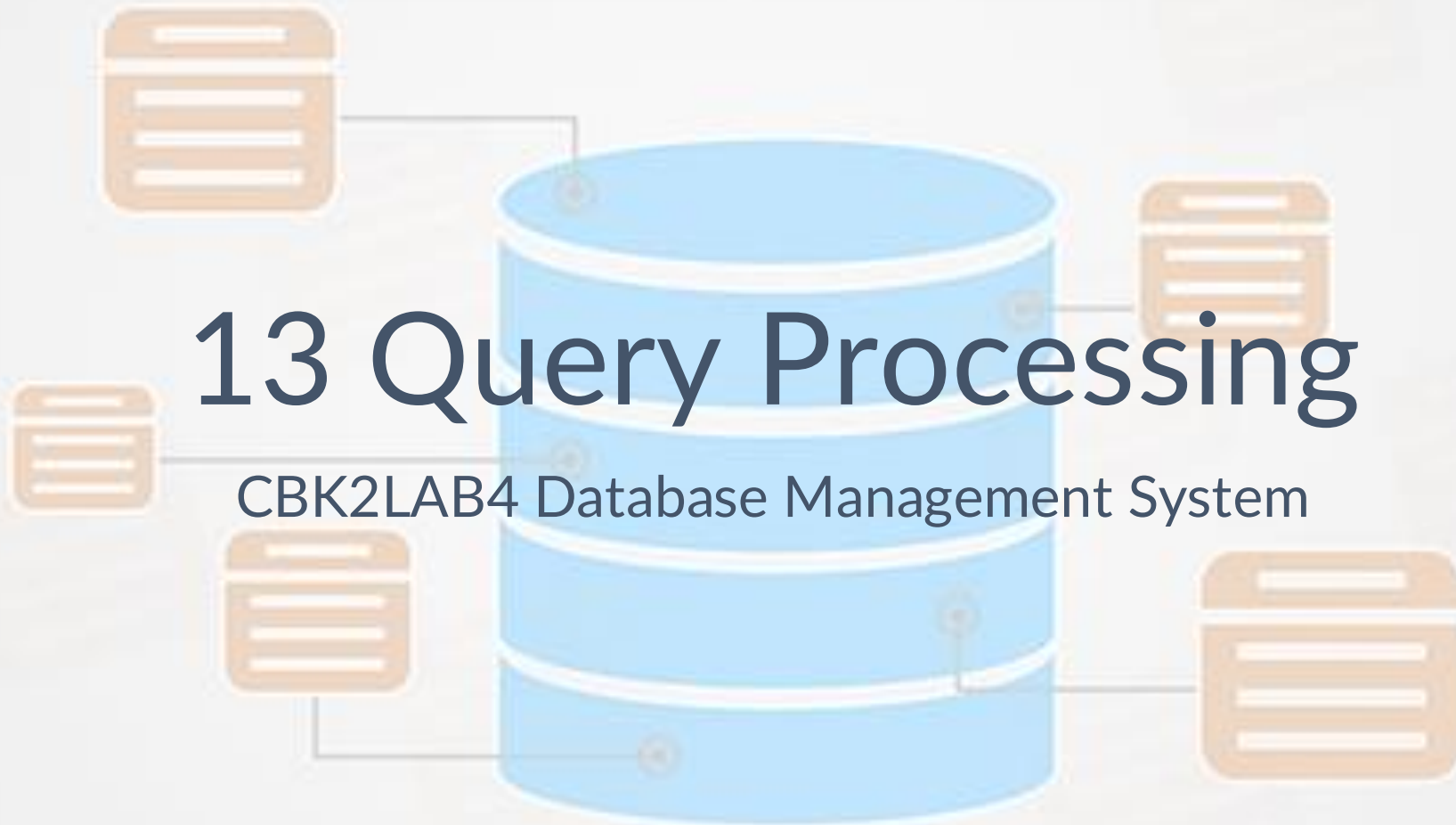


13 Query Processing

CBK2LAB4 Database Management System



Sub-CLO-04-1-2

Students are able to identify data storage management, indexing, query processing in database systems.

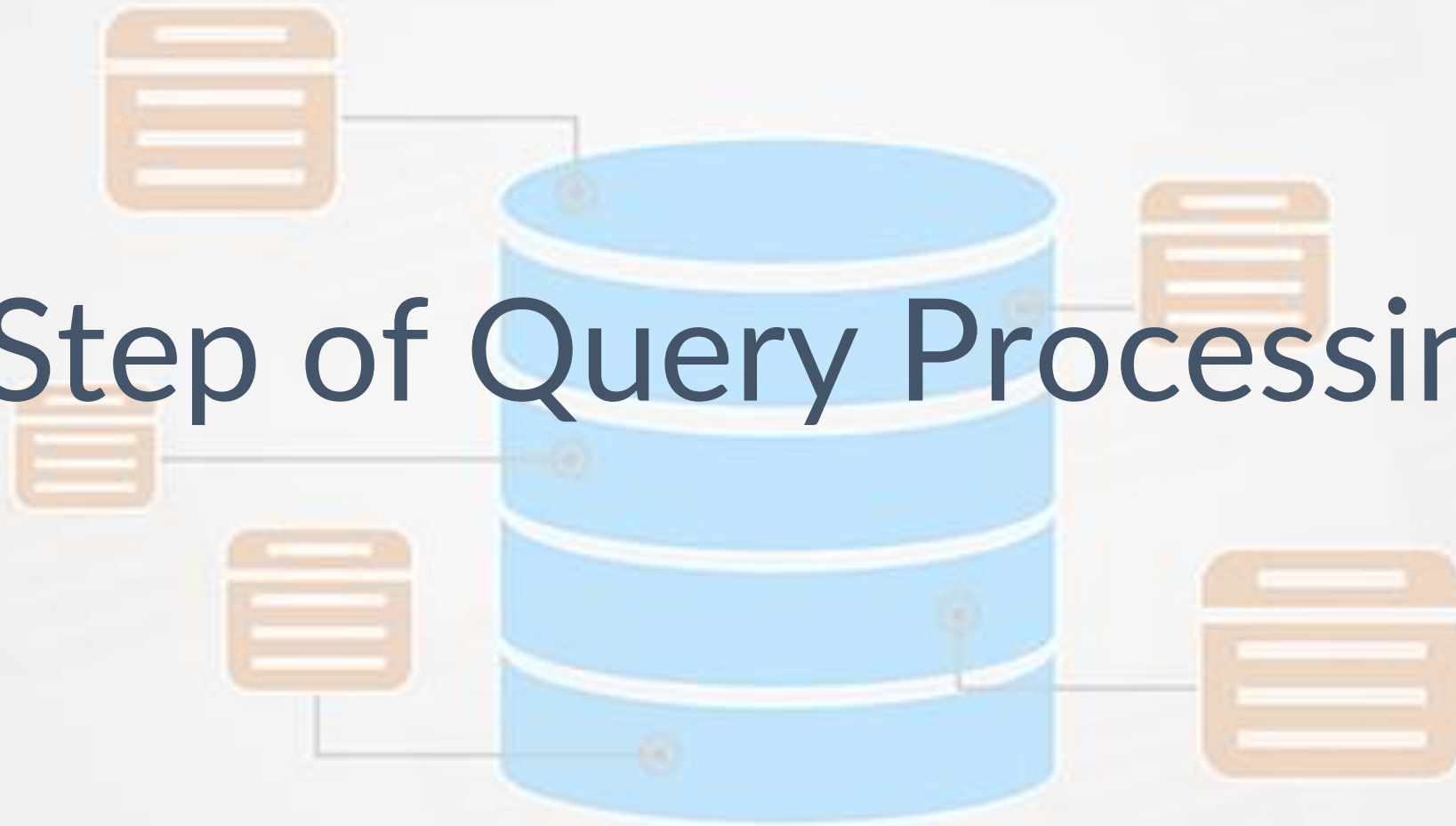
Topics

Step of Query
Processing

Relational
Algebra
Expression (RAE)

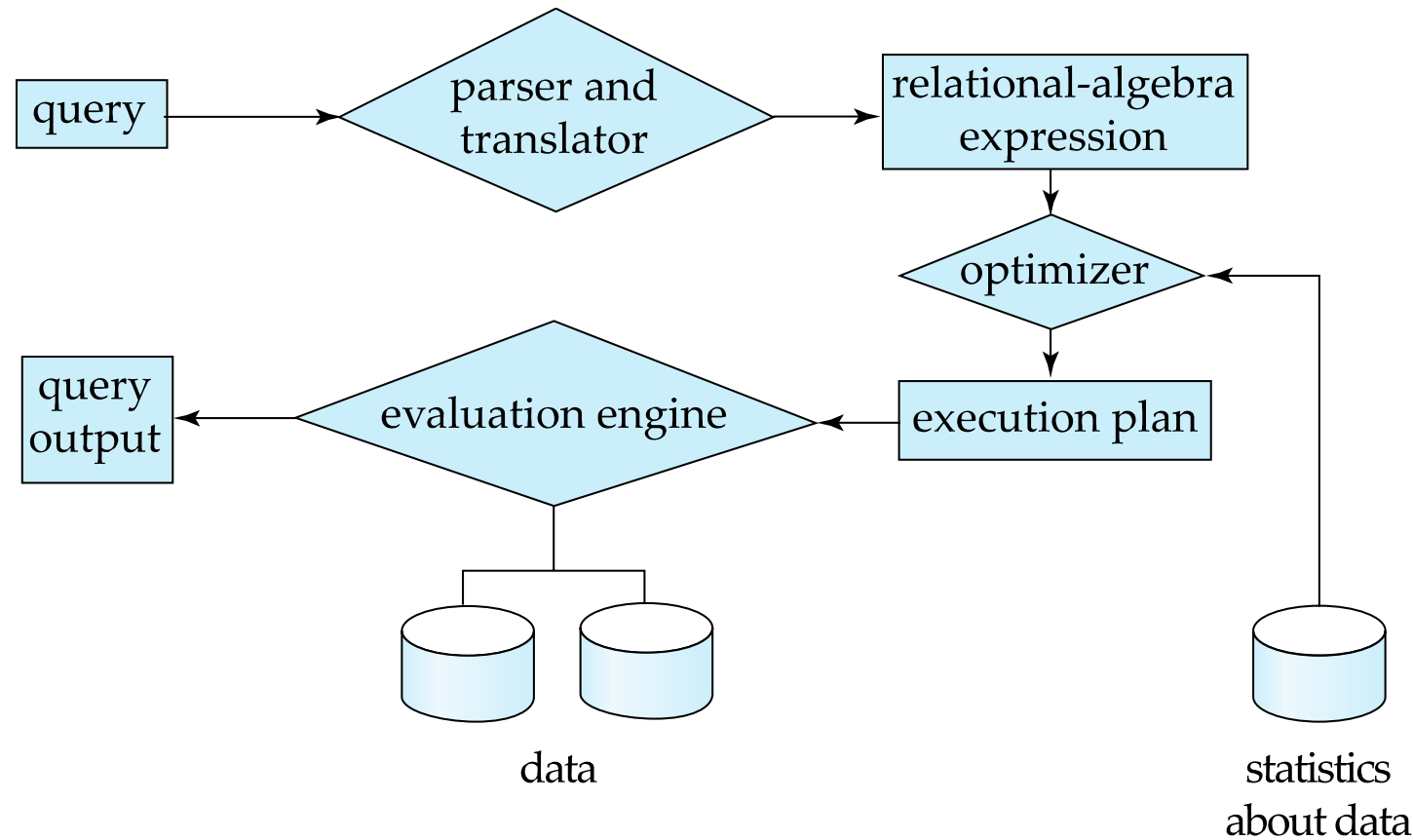


Step of Query Processing



Basic Steps in Query Processing

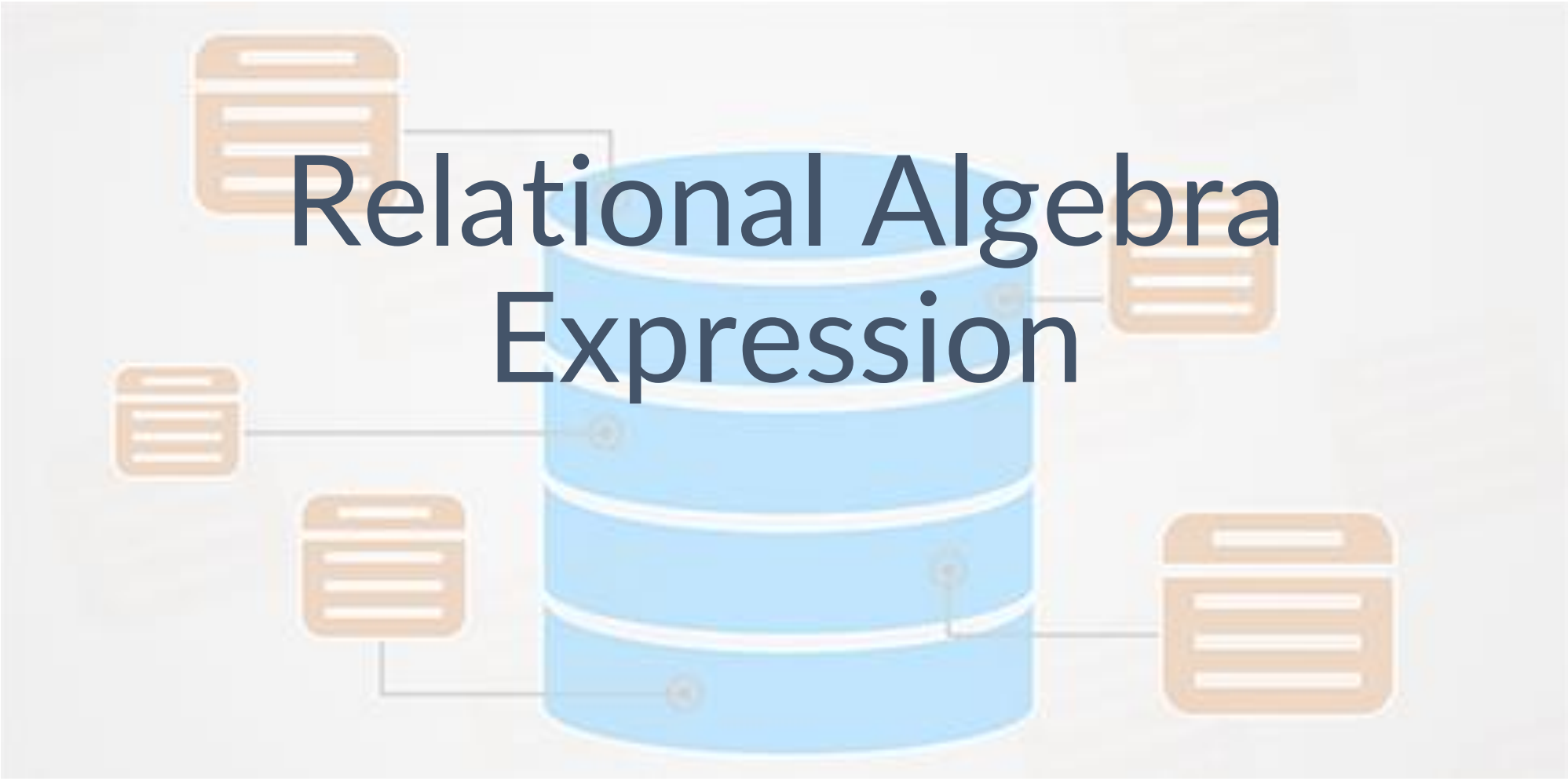
1. Parsing and translation
2. Optimization
3. Evaluation



Basic Steps in Query Processing (Cont.)

- Parsing and translation
 - translate the query into its internal form. This is then translated into relational algebra.
 - Parser checks syntax, verifies relations
 - Optimization
 - Each relational algebra operation can be evaluated using one of several different algorithms
 - Annotated expression specifying detailed evaluation strategy is called an evaluation-plan
 - Amongst all equivalent evaluation plans choose the one with lowest cost.
 - Evaluation
 - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.
- 

Relational Algebra Expression

A central blue cylinder representing a database is connected by lines to five orange document icons with horizontal lines, symbolizing data retrieval or processing. The entire scene is set against a light gray background with a subtle grid.

Relational Algebra

- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.
 - Operators
 - select: σ
 - project: Π
 - cartesian product: $\times$
 - join: $\bowtie$
 - assignment: $\leftarrow$
 - rename: ρ
- 

Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the “Physics” department.
 - Query
SELECT * FROM instructor WHERE dept_name = ‘Physics’
 - Relational Algebra (RA)

$\sigma_{dept_name="Physics"}(instructor)$

- Result

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation

- We allow comparisons using

$=, \neq, >, \geq, <, \leq$

in the selection predicate.

- We can combine several predicates into a larger predicate by using the connectives:

$\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

- Example: Find the instructors in Physics with a salary greater than \$90,000, we write:

- Query: **SELECT * FROM instructor WHERE dept_name = 'Physics' AND salary > 90000**
- RA:

$\sigma_{dept_name='Physics' \wedge salary > 90,000} (instructor)$

- The select predicate may include comparisons between two attributes.
 - Example, find all departments whose name is the same as their building name:
 - Query: **SELECT * FROM department WHERE dept_name = building**
 - RA:

$\sigma_{dept_name=building} (department)$

Project Operation

- A unary operation that returns its argument relation, with certain attributes left out.
- Notation:

$$\Pi_{A_1, A_2, A_3 \dots A_k}(r)$$

where $A_1, A_2, \dots, A_k$ are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets

Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*
- Query: **SELECT** id, name, salary **FROM** instructor
- RA:

$\Pi_{ID, name, salary} (instructor)$

- Result:

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Cartesian-Product Operation

- The Cartesian-product operation (denoted by $\times$) allows us to combine information from any two relations.
- Example: the Cartesian product of the relations *instructor* and *teaches* is written as:
 - Query: **SELECT * FROM instructor CROSS JOIN teaches**
or **SELECT * FROM instructor, teaches**
 - RA:
instructor $\times$ *teaches*
- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation (see next slide)
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
 - *instructor.ID*
 - *teaches.ID*

[illegible]

Join Operation

- The Cartesian-Product

instructor X teaches

associates every tuple of instructor with every tuple of teaches.

- Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught, we write:
 - Query: **SELECT * FROM instructor, teaches WHERE instructor.id = teaches.id**
 - RAE:

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$
- We get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught.
- The result of this expression, shown in the next slide

Join Operation (Cont.)

- The table corresponding to:

$\sigma_{instructor.id = teaches.id}$ (*instructor x teaches*)

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Join Operation (Cont.)

The join operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.

Consider relations r (R) and s (S)

Let “theta” be a predicate on attributes in the schema R “union” S . The join operation $r \bowtie_{\theta} s$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

Thus

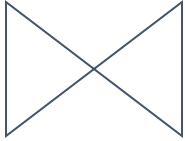
$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$

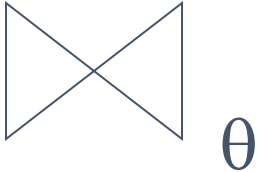
Can equivalently be written as

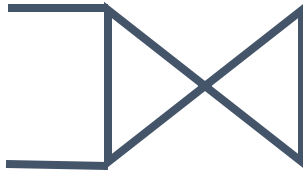
SELECT * FROM instructor JOIN teaches ON instructor.id = teaches.id

$$instructor \bowtie_{Instructor.id = teaches.id} teaches$$

Join Operation (Cont.)

- Natural join : 

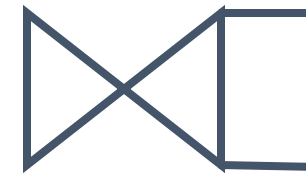
- Inner join : 

- Outer join : 

Left Outer Join



Full Outer Join



Right Outer Join

The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.
- Example: Find all instructor in the “Physics” and Music department.

Physics $\leftarrow \sigma_{dept_name="Physics"}(instructor)$

Music $\leftarrow \sigma_{dept_name="Music"}(instructor)$

Physics $\cup$ *Music*

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.

Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ , is provided for that purpose

- The expression:

$$\rho_x(E)$$

returns the result of expression E under the name x

- Another form of the rename operation:

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

returns the result of expression E under the name x , and with the attributes renamed to $A_1, A_2, \dots, A_n$.

Rename Operation Example

- SELECT * FROM countries nation:

$\rho_{nation}(\text{countries})$

- SELECT country_id AS id, country_name AS name, region_id FROM countries nation:

$\rho_{nation}(id, name, region_id)(\text{countries})$

- SELECT country_id AS id, country_name AS name FROM countries nation:

$\Pi_{id, name}(\rho_{nation}(id, name, region_id)(\text{countries}))$

Reference

Silberschatz, Korth, and Sudarshan. *Database System Concepts* – 7th Edition.
McGraw-Hill. 2019.

Slides adapted from Database System Concepts Slide.

Source: <https://www.db-book.com/db7/slides-dir/index.html>



Questions

